Renox: Dokumen Arsitektur Platform

Oct 26, 2026 · @Achmad Arif Rachim

Versi 2. Ditulis ulang setelah tinjauan kritis: tujuan diperjelas menjadi platform hosting multi-tenant, runtime dibangun di atas Supabase Edge Runtime alih-alih `deno_core` mentah, data tenant memakai SQLite per tenant, dan render template memakai minijinja.

1. Ringkasan dan tujuan

Renox adalah platform hosting multi-tenant: pelanggan (tenant) men-deploy aplikasi web dan function dalam TypeScript, dan Renox menjalankannya dengan aman, murah, dan cepat di infrastruktur milik kita. Pendapatan berasal dari biaya hosting per tenant, sehingga dua hal menentukan bisnisnya: **kepadatan** (berapa tenant per server) dan **isolasi** (tenant tidak bisa saling mengganggu atau membaca data tenant lain).

Platform terdiri dari dua bagian yang dipisahkan tegas:

- **Data plane**: binary Rust yang menjalankan kode tenant di dalam isolate V8, dibangun di atas Supabase Edge Runtime. Ini satu-satunya bagian yang membenarkan Rust, karena sandbox per tenant tidak tersedia di runtime lain.
- **Control plane**: dashboard, API deploy, billing, registry modul. Kode milik kita sendiri, tidak butuh sandbox, dan ditulis dengan Bun supaya cepat dikembangkan.

Aplikasi tenant mengikuti satu model: SSR HTML-first (template atribut + htmx), function/API, modul yang bisa dirakit seperti Lego (auth, billing, admin), dan data di SQLite per tenant.

Tujuan

1. **Kepadatan tinggi.** Target: ribuan tenant idle per server 8 GB, dengan biaya tenant idle mendekati nol (isolate dibuang, file SQLite ditutup).
2. **Isolasi berlapis.** Isolate V8 + proses sandbox + file data per tenant. Satu bug di satu lapisan tidak boleh membocorkan data.
3. **Development cepat bagi tenant.** Deploy dari save ke live dalam hitungan detik, tanpa Node, tanpa build step di sisi tenant, dengan modul siap pakai.
4. **Runtime kencang.** Static file, routing, render HTML, dan query dikerjakan di luar kuota tenant oleh Rust dan SQLite in-process.
5. **Billing akurat.** Setiap request dimeter: CPU, wall-clock, memory, egress, render.

Non-goals (versi awal)

- Menjalankan Next.js atau framework yang bergantung penuh pada Node API internal.
- Multi-region. Target awal satu region, beberapa node worker.
- Tenant berukuran enterprise dengan beban tulis sangat tinggi. Untuk mereka disediakan jalur upgrade ke Postgres, bukan dioptimalkan sejak awal.
- Marketplace modul publik dengan pembayaran. Tahap awal: modul resmi dan modul dari git.

2. Posisi produk

Pengguna tidak memilih platform karena runtime-nya. Mereka memilih karena ekosistem, kecepatan sampai ke produk, risiko jangka panjang, dan biaya. Karena itu Renox dijual sebagai hasil, bukan teknologi: **"Rakit SaaS dari modul auth, billing, dan admin, lalu deploy ke hosting yang biayanya mengikuti pemakaian."** Kalimat itu tidak menyebut Rust, dan memang tidak perlu.

Celah yang diisi

Kebutuhan	Next.js	Laravel / Rails	Supabase	PocketBase	Bun + Hono	Renox
Logic dalam TypeScript	Ya	Tidak	Ya (edge fn)	JS hooks	Ya	Ya
SSR HTML-first, tanpa SPA	Tidak	Ya	Tidak	Tidak	Manual	Ya
Auth, admin, billing bawaan	Tidak	Ya	Sebagian	Ya	Tidak	Ya (modul)
Modul yang bisa dirakit	Tidak	Ya	Tidak	Tidak	Tidak	Ya
Hosting kode pihak ketiga dengan sandbox	Tidak	Tidak	Ya	Tidak	Tidak	Ya
Biaya tenant idle mendekati nol	Tidak	Tidak	Sebagian	Ya	Tidak	Ya

Kombinasi "TypeScript + batteries-included + modul + hosting murah" tidak ada. Di PHP/Ruby/Python kombinasi itu sudah menang (Laravel, Rails, Django, Odoo); di TypeScript belum ada yang berhasil. PocketBase (satu binary Go, SQLite) membuktikan permintaan untuk "satu binary + batteries".

Kenapa orang meninggalkan Next.js, dan kenapa mereka bertahan

Alasan pergi: self-hosting sulit dan pengalaman terbaik hanya di Vercel, churn model mental (Pages → App Router → RSC → Server Actions), bukan backend sehingga auth/ORM/queue/billing dirakit ulang setiap project, memory produksi ratusan MB, dan

kelelahan JavaScript yang mendorong sebagian developer ke HTMX, Astro, Rails, dan Laravel.

Alasan bertahan: React adalah bahasa UI mayoritas, lowongan kerja, dokumentasi, dan perusahaan besar di belakangnya. Renox tidak melawan ini secara frontal. Renox bermain di celah yang Next.js tidak isi: aplikasi bisnis lengkap yang dirakit dari modul dan dijalankan murah.

Pelajaran dari framework yang gugur

RedwoodJS, Blitz.js, Meteor, Sails, Wasp, dan Deno Fresh mencoba celah serupa dan tidak lepas landas. Polanya sama: framework dibuat dulu, aplikasi dicari kemudian. Framework yang berhasil arahnya terbalik: Rails diekstrak dari Basecamp, Laravel dari aplikasi Taylor Otwell, Django dari sistem redaksi koran, PocketBase dari kebutuhan pembuatnya sendiri.

Target pengguna

- Solo developer dan agency kecil yang membangun banyak aplikasi bisnis dan lelah merakit auth, admin, dan billing berulang.
- Pasar yang sensitif biaya (Indonesia dan Asia Tenggara): aplikasi harus jalan murah, bukan di Vercel dengan tagihan dolar.
- Developer yang menyukai kelengkapan Laravel tetapi ingin TypeScript.

Syarat supaya tidak sia-sia

1. **Dogfooding.** Minimal dua produk nyata dibangun dan dihost di Renox sejak awal. Kalau platformnya gagal, produknya tetap ada.
2. **Nilai ada di perakitan, bukan komponen.** Runtime, template engine, policy engine, dan parser dipakai dari yang sudah ada. Usaha dipusatkan ke control plane, modul, dan konvensi.
3. **Kriteria berhenti ditetapkan di awal** (bagian 16). Kalau tidak tercapai, kembali ke stack biasa tanpa rasa bersalah.

3. Keputusan arsitektur

Setiap keputusan diambil untuk kepadatan, isolasi, dan kecepatan development. Keputusan yang berubah dari versi 1 ditandai, supaya alasannya bisa ditinjau ulang kalau kondisi berubah.

#	Keputusan	Alasan	Alternatif yang ditolak
D1	Tujuan: platform hosting kode pihak ketiga, bukan framework untuk aplikasi sendiri	Sandbox multi-tenant adalah inti produk dan sumber pendapatan. Keputusan ini yang membenarkan semua keputusan teknis di bawah	Framework TS di atas Bun untuk produk sendiri (lebih murah, tapi tanpa sandbox dan tanpa bisnis hosting)
D2	Data plane ditulis dengan Rust	Kontrol penuh atas isolate V8: heap limit, watchdog CPU, izin per op, scheduling. Tidak tersedia di Bun/Node	Bun proses per tenant (30–50 MB per tenant, 5–10x kurang padat)
D3 (berubah)	Runtime dibangun di atas Supabase Edge Runtime / `deno_runtime`, bukan `deno_core` mentah	Web API, `npm:`, `node:*`, source map, batas memory/CPU per worker sudah ada dan memang dibuat untuk menjalankan function pelanggan. Memangkas scope terbesar	`deno_core` mentah (harus membangun ulang kompatibilitas Node), QuickJS (10–50x lebih lambat), Bun (tanpa isolate)
D4 (baru)	Control plane ditulis dengan Bun	Kode milik kita, tidak butuh sandbox, dan kecepatan development jauh lebih penting di sini	Rust untuk semuanya (lambat dikembangkan untuk CRUD dan dashboard)

#	Keputusan	Alasan	Alternatif yang ditolak
D5 (berubah)	Data tenant di SQLite per tenant; control plane di Postgres	Isolasi per file lebih kuat dari RLS, biaya tenant idle nol, query in-process mikrodetik, backup dan pindah tenant trivial. Model yang dipakai Cloudflare D1, Turso, Fly.io, PocketBase	Postgres schema per tenant (catalog bloat, RLS sebagai satu-satunya pagar), Postgres DB per tenant (koneksi mahal, restore per tenant sulit)
D6	Isolasi berlapis: isolate V8 + proses sandbox (seccomp, namespace, cgroup) + tier proses	Isolate saja tidak cukup untuk kode yang tidak dipercaya (side-channel, bug V8)	Isolate saja (Cloudflare pun tidak melakukannya), microVM untuk semua (terlalu mahal untuk long tail)
D7	Render HTML di Rust (minijinja), JS hanya mengembalikan data; jalur alternatif template-ke-JS disediakan dan dipilih lewat benchmark	Render di luar kuota tenant, heap tenant tetap kecil, template adalah DSL terbatas tanpa eksekusi kode. Biaya marshaling V8→Rust nyata dan harus diukur	SSR penuh di JS sebagai satu-satunya jalur
D8 (berubah)	Sintaks template berbasis atribut HTML, di-transpile ke minijinja saat deploy	Template tetap HTML valid dan menyatu dengan htmx/Alpine; performa, escaping, filter, dan render per block mengikuti minijinja yang teruji. Bagian custom hanya penerjemah sintaks	Engine render sendiri (bug dan fitur yang kurang), Jinja langsung (sintaks blok tidak disukai)
D9	Interaktivitas lewat fragment HTML + htmx + idiomorph + View Transitions	Partial update tanpa SPA, state DOM dan animasi terjaga, JS client kecil	React/SPA

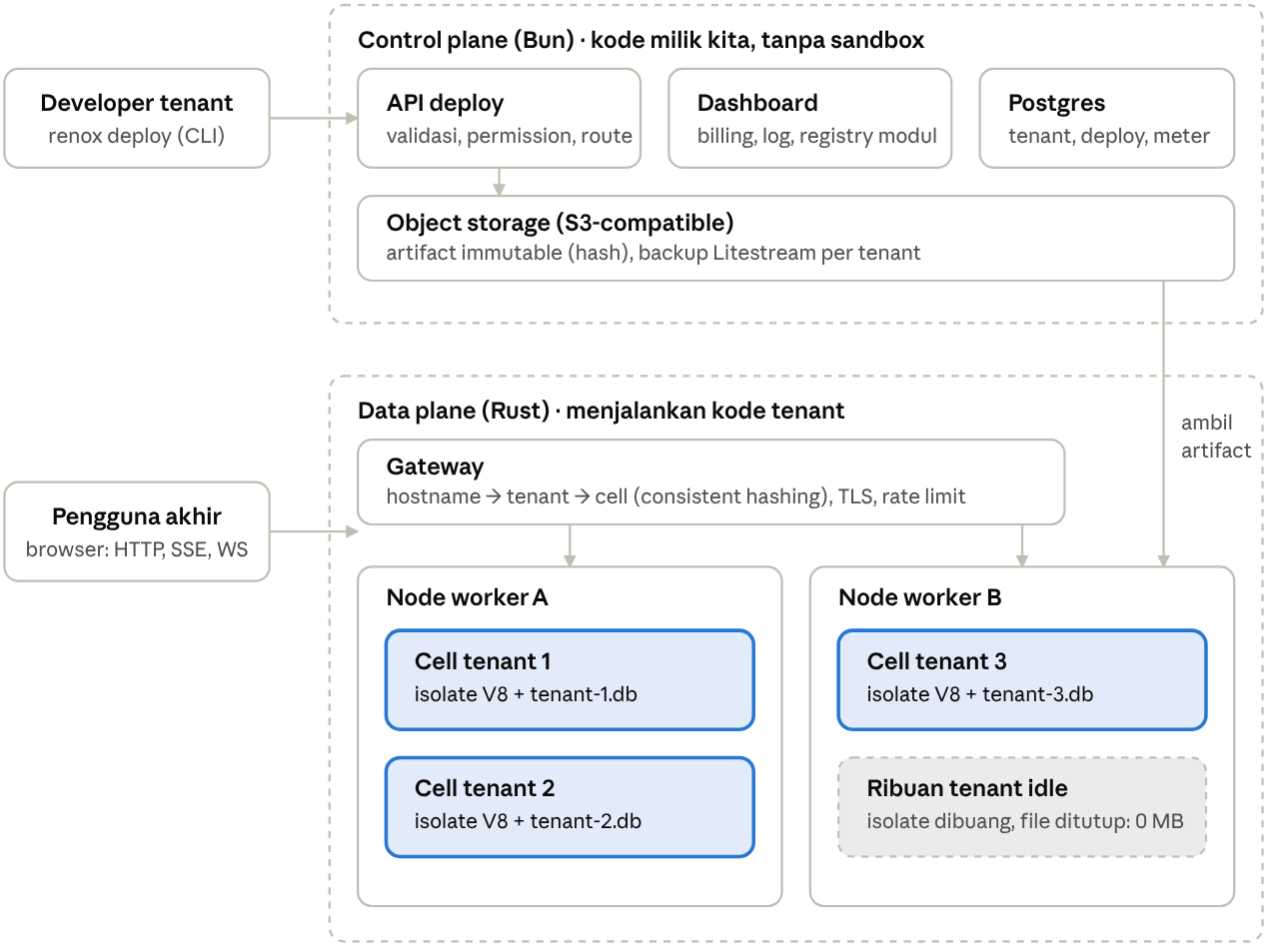
#	Keputusan	Alasan	Alternatif yang ditolak
D10	Convention over configuration untuk routing	Tanpa boilerplate router, struktur folder = struktur URL	Router manual di config
D11	Modul = mini app dengan struktur folder yang sama; modul resmi (auth, billing, admin) menjadi produk	Satu konsep, dan modul adalah alasan utama tenant memilih Renox	Plugin API khusus
D12 (berubah)	Mode isolasi modul: `shared` untuk modul milik tenant sendiri, `isolate` untuk modul marketplace	Modul sendiri tidak perlu serialisasi antar context; modul orang lain tidak boleh menyentuh heap tenant	`context` sebagai default untuk semua (overhead serialisasi tanpa manfaat keamanan nyata)
D13 (berubah)	Tabel per modul memakai prefix (`billing__invoices`) dalam satu file tenant; izin per modul lewat SQLite authorizer	Transaksi lintas modul tetap atomic (ATTACH lintas file tidak atomic dalam mode WAL); authorizer memeriksa akses tabel saat statement di-compile oleh SQLite sendiri	File per modul lewat ATTACH, parsing SQL di Rust
D14 (berubah)	Durable event lewat outbox di file tenant, worker dibangunkan oleh `update_hook`	Event dan data dalam satu transaksi; tidak perlu polling ribuan file	Kafka/NATS, LISTEN/NOTIFY Postgres
D15	Otorisasi user dengan Cedar (`cedar-policy`): RBAC + ABAC + relasi	Native Rust, policy di-load saat runtime, bisa divalidasi terhadap schema, `forbid` menang atas `permit`	OPA/Rego (sidecar Go), Casbin, predicate TS

#	Keputusan	Alasan	Alternatif yang ditolak
D16 (berubah)	Tanpa RLS; pengaman per baris lewat `db.scoped()` + Cedar + test otorisasi wajib	SQLite tidak punya RLS; isolasi tenant sudah ditangani file per tenant. Kehilangan pagar per baris diakui sebagai trade-off	Mempertahankan Postgres hanya demi RLS
D17 (baru)	Migration forward-only dengan pola expand/contract; public view diberi versi	Rollback `down` lintas modul merusak dependensi view; migration maju yang kompatibel lebih aman	Migration up/down klasik
D18 (baru)	Satu dialek SQL untuk ekosistem modul: SQLite	Modul yang mendukung dua dialek diuji dua kali dan penulis pihak ketiga hanya akan mendukung salah satu	Dual dialect
D19 (baru)	Hot reload di dev dengan membuang isolate dan membuat yang baru	Lebih sederhana dan bebas memory leak daripada mengganti context di tempat; puluhan ms masih dapat diterima di dev	Hot-swap context per modul
D20 (baru)	MVP dipangkas: F0–F2 + modul auth + dua produk nyata, dengan kriteria berhenti	Framework yang dibangun tanpa produk hampir selalu sia-sia	Membangun seluruh roadmap sebelum ada pengguna

4. Arsitektur sistem

Platform dibagi menjadi control plane (mengelola) dan data plane (menjalankan). Keduanya hanya berbagi dua hal: Postgres control plane sebagai sumber kebenaran, dan object storage sebagai tempat artifact dan backup.

Control plane mengelola deploy dan billing, data plane menjalankan cell tenant



arsitektur renox v2 · control plane, data plane, cell per tenant

Developer tenant berbicara ke control plane; pengguna akhir berbicara ke data plane. Node worker mengambil artifact dari object storage dan mengirim meter serta log kembali ke control plane.

Komponen	Tanggung jawab	Teknologi
API deploy	Menerima bundle, validasi, menyimpan artifact, mengaktifkan route	Bun, Hono
Dashboard	Tenant, deployment, log, billing, registry modul	Bun, SSR Renox sendiri (dogfooding)
Postgres control plane	Tenant, deployment, permission, secret terenkripsi, meter, audit	Postgres
Object storage	Artifact immutable per hash, backup Litestream per tenant	S3-compatible

Komponen	Tanggung jawab	Teknologi
Gateway	TLS, routing hostname → tenant → cell, rate limit, route table di memory	Rust: `hyper`, `axum`, `rustls`
Cell tenant	Isolate V8 per deployment, file SQLite tenant, koneksi realtime, outbox worker	Rust: Supabase Edge Runtime, `rusqlite`
Template renderer	Transpile atribut → minijinja saat deploy; render di luar kuota tenant	Rust: `html5ever`, `minijinja`
Policy engine	Evaluasi Cedar per request	Rust: `cedar-policy`
Metering	Record per invocation → agregasi per jam	Rust (emit), Postgres (agregasi)

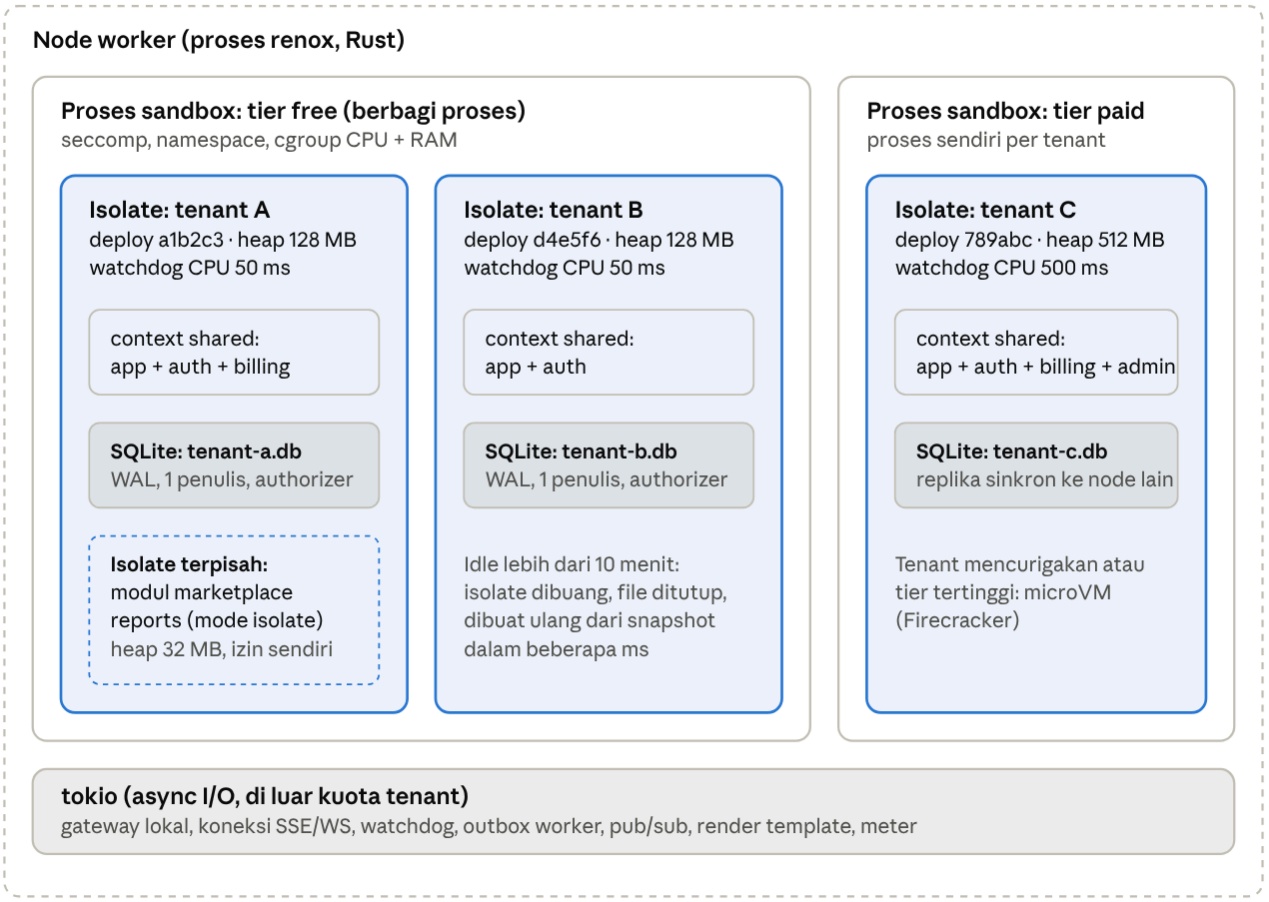
Jalur request SSR

1. Gateway mencocokkan hostname dan path dengan route table deployment tenant.
2. Request diarahkan ke cell tenant. Kalau isolate belum ada, dibuat dari snapshot dan artifact.
3. `guard` route dievaluasi oleh policy engine.
4. Loader dijalankan di isolate; query ke SQLite lewat op Rust dengan authorizer modul.
5. Data di-marshal ke Rust, template direnderer (penuh atau fragment), response di-stream.
6. Meter dicatat: CPU, wall, heap, egress, render.

5. Model eksekusi dan isolasi multi-tenant

Setiap tenant mendapat satu isolate V8 per deployment, di dalam proses yang di-sandbox oleh OS, dengan file SQLite sendiri. Tidak ada runtime baru per request atau per panggilan antar modul. Isolate dibuat dari snapshot saat request pertama, dipakai ulang selama tenant aktif, dan dibuang saat idle.

Tiga lapis isolasi per tenant: proses sandbox, isolate V8, file SQLite



model isolasi · proses sandbox per tier, isolate per tenant, context per modul, file per tenant

Satu node menampung banyak isolate dalam proses tier free dan beberapa proses terpisah untuk tier paid. Semua I/O, koneksi realtime, render, dan metering dikerjakan tokio di luar kuota tenant.

Hierarki dan biaya

Lapisan	Kapan dibuat	Perkiraan biaya	Batas yang ditegakkan
Proses sandbox	Per tier (free berbagi; paid per tenant)	Puluhan MB per proses	cgroup CPU/RAM, seccomp, namespace
Isolate V8	Per tenant per deployment, saat request pertama	~2–10 MB idle, dari snapshot dalam ms	Heap limit, watchdog CPU, izin op (fetch, db, secret)
Context modul	Mode `shared`: satu; mode `isolate`: isolate sendiri	Ratusan KB	Authorizer tabel, allowlist network

Lapisan	Kapan dibuat	Perkiraan biaya	Batas yang ditegakkan
Request	Setiap request	Pemanggilan fungsi	Deadline, semaphore per deployment
File SQLite	Dibuka bersama isolate, ditutup saat idle	Cache page sesuai konfigurasi	Authorizer, `busy_timeout`, batas statement

Angka memory dan cold start adalah target yang harus dibuktikan di F0 (bagian 16).

Konkurensi

- **Event loop per isolate.** Saat handler `await db.query(...)`, op Rust mengerjakannya dan isolate melayani request lain. Satu isolate bisa menangani puluhan request yang menunggu I/O.
- **Kode CPU-bound memblokir isolate itu saja**, bukan tenant lain. Watchdog menghentikan request yang melewati batas CPU, lalu isolate di-recycle supaya tidak ada state rusak.
- **Scheduler** tidak mengirim request baru ke thread yang sedang menjalankan isolate sibuk; `ctx.offload(fn)` menjalankan pekerjaan berat di pool terpisah dengan kuota sendiri.
- **Penempatan**: request tenant yang sama selalu ke node yang sama (consistent hashing) supaya isolate warm dan file SQLite lokal.
- **State global di context dipakai ulang lintas request.** Data per request wajib lewat `ctx`; mode dev memberi peringatan saat handler menulis global.

Snapshot dan cold start

Snapshot platform (Web API, polyfill Node, SDK Renox) dibuat sekali per rilis. Saat deploy, kode tenant di-bundle sebagai eszip dengan V8 code cache. Cold start = snapshot + load eszip, target beberapa hingga puluhan milidetik. Kalau cold start terbukti lebih lambat, tenant tier paid bisa dipertahankan warm.

Defense in depth (D6)

Lapisan	Melindungi dari
Isolate + heap limit + watchdog	Tenant memakan CPU/memory tenant lain
`SharedArrayBuffer` dimatikan, timer dikasarkan	Side-channel (Spectre) antar isolate

Lapisan	Melindungi dari
Proses sandbox (seccomp/Landlock, namespace, cgroup)	Bug V8 yang memungkinkan escape dari isolate
Tier proses: free berbagi, paid proses sendiri, mencurigakan → microVM	Blast radius saat satu lapisan jebol
File SQLite per tenant + authorizer	Query yang menyeberang tenant atau modul
Bundle immutable, tanpa akses npm/filesystem di runtime	Supply chain setelah deploy
Blok IP privat dan metadata endpoint	SSRF ke infrastruktur internal

Prinsipnya: asumsikan setiap lapisan suatu saat bocor, dan pastikan lapisan berikutnya menahan. Cloudflare dan Deno Deploy memakai pelapisan yang sama.

6. Siklus hidup kode tenant

Kode tenant menjadi artifact immutable yang dirujuk lewat hash. Tidak ada build, instalasi npm, atau akses filesystem di runtime; semua dependency dibekukan saat deploy.

Pipeline deploy

1. `renox deploy` mem-bundle kode: TS → JS (swc), semua import `npm:` dan `node:` diresolve, hasilnya satu arsip eszip + manifest + template yang sudah di-transpile ke minijinja + policy Cedar.
2. CLI mengunggah ke control plane (`POST /v1/deployments`) dengan token tenant.
3. Control plane memvalidasi: ukuran bundle, manifest, permission yang diminta (network, secret, modul), template dan policy terhadap schema, konflik route.
4. Artifact disimpan di object storage (S3-compatible), content-addressed: `sha256(bundle)`. Artifact lama tidak pernah ditimpa.
5. Deployment dicatat di Postgres control plane: tenant, hash, permission yang disetujui, versi secret.
6. Route table di gateway diperbarui lewat pub/sub: hostname tenant → hash baru. Rollback = pindahkan pointer ke hash lama.

Jalur request

1. Gateway memetakan hostname → tenant → deployment (cache di memory, invalidasi lewat pub/sub).

2. Request diarahkan ke node worker yang memegang cell tenant (consistent hashing), supaya isolate dan file SQLite-nya ada di satu tempat.
3. Kalau isolate belum ada (cold start): worker mengambil artifact dari cache disk atau object storage, memverifikasi hash, membuat isolate dari snapshot platform, lalu memuat modul tenant dengan V8 code cache. Target beberapa hingga puluhan milidetik.
4. Isolate tetap warm. Setelah idle melewati batas (misalnya 10 menit), isolate dibuang dan file SQLite ditutup. Ini scale-to-zero per tenant.

Batas per tenant

Batas	Mekanisme	Contoh default
CPU per request	Watchdog thread → `TerminateExecution`, isolate di-recycle setelahnya	50 ms (free), 500 ms (paid)
Wall-clock	Deadline di context, I/O dibatalkan	30 detik
Memory	Heap limit V8 per isolate, cgroup per proses	128 MB
Request bersamaan	Semaphore per deployment	50
Network keluar	Allowlist di op `fetch`, blok IP privat dan metadata endpoint cloud (anti-SSRF), batas egress	Hanya host yang dideklarasikan
Filesystem	Tidak ada; hanya API storage platform	—
Bundle dan versi	Validasi saat deploy	20 MB, 50 versi disimpan
Query	Pool kecil per tenant, `busy_timeout`, batas waktu statement	10 detik
Render	Fuel minijinja, batas iterasi dan ukuran output	Dihitung ke kuota tenant

Secret dan data

- Secret terenkripsi (envelope encryption), diputar per versi, disuntikkan ke isolate sebagai object beku lewat op Rust. Tidak pernah masuk bundle, log, atau pesan error.
- Tenant hanya menerima `db` yang sudah terikat ke file miliknya. Tidak ada path atau koneksi mentah di sisi tenant.

Metering dan billing

Setiap invocation menghasilkan satu record: `tenant, deployment, function, cpu_ms, wall_ms, peak_heap, egress_bytes, db_ms, render_fuel`. Record diagregasi per jam ke tabel billing di Postgres. Log `console.log` tenant di-stream ke sink per tenant dengan batas volume, dapat dilihat di dashboard atau `renox logs --follow`.

Modul marketplace

- Publisher menandatangani modul (Ed25519); registry menyimpan hash dan tanda tangan; tenant hanya bisa memasang modul yang verifikasinya lolos.
- Permission di-review saat install ("minta akses ke `api.stripe.com`, tabel `billing__*`, slot `admin.sidebar`") dan diff permission harus disetujui lagi saat upgrade.
- Modul marketplace berjalan dalam mode `isolate`, tidak melihat secret tenant kecuali yang dideklarasikan dan disetujui.
- Hash modul yang terbukti berbahaya bisa dinonaktifkan secara global, dan tenant yang memakainya diberi tahu.

7. Data

Data tenant disimpan di SQLite, satu file per tenant, di node yang sama dengan isolate-nya. Data platform (tenant, deployment, billing, audit) disimpan di Postgres. Pemisahan ini mengikuti Cloudflare (D1 untuk tenant, Postgres internal) dan Turso.

Lapisan	Database	Alasan
Control plane	Postgres	Query lintas tenant, analitik, transaksi billing, tooling matang
Data tenant, default	SQLite per tenant (`rusqlite`/libSQL), WAL, Litestream ke S3	Isolasi per file, biaya idle nol, latency mikrodetik, backup per tenant
Data tenant, tier tinggi	SQLite dengan replika sinkron (libSQL/LiteFS) ke node kedua	Failover otomatis, RPO mendekati nol
Tenant besar	Postgres managed atau BYO	Beban tulis tinggi, ukuran ratusan GB

Kenapa SQLite per tenant

Aspek	SQLite per tenant	Postgres schema per tenant
Isolasi tenant	Satu file per tenant; tidak ada query yang bisa menyeberang	RLS/search_path; satu bug = bocor
Biaya tenant idle	Nol (file tertutup)	Catalog bloat pada puluhan ribu schema
Latency query	Mikrodetik, in-process; N+1 di loader tidak fatal	Ratusan µs–ms per round-trip
Backup/restore per tenant	Copy file, point-in-time per tenant	Sulit memulihkan satu tenant
Pindah tenant antar node	Copy file	Dump/restore
Write concurrency	Satu penulis per file (WAL: banyak pembaca)	Tinggi
Dev lokal	Zero setup, engine sama dengan produksi	Harus menjalankan Postgres

Struktur di dalam file tenant

- **Prefix tabel per modul**: `auth__users`, `billing__invoices`. SQLite tidak punya schema, dan `ATTACH` lintas file tidak atomic dalam mode WAL, sehingga satu file per tenant dipilih supaya transaksi lintas modul tetap atomic.
- **Izin per modul lewat `sqlite3_set_authorizer`**: Rust memasang callback per koneksi modul yang hanya mengizinkan `billing__*` dan baca ke view `*__public_*`. Pemeriksaan dilakukan SQLite saat statement di-compile, tanpa parsing SQL di Rust.
- **Tabel platform di file tenant**: `renox__migrations`, `renox__outbox`, `renox__sessions`, `renox__role_assignments`.
- **Migration** per modul, forward-only, pola expand/contract. Checksum dicatat supaya file migration lama tidak diubah diam-diam.

Pengaturan koneksi

- Satu koneksi penulis + pool pembaca kecil per tenant, `journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout`, prepared statement cache.
- Koneksi tidak dibagi antar thread secara bersamaan; pool dikelola Rust di thread worker cell.

- Checkpoint WAL dijadwalkan supaya file WAL tenant sibuk tidak membengkak.
- Koneksi tenant idle ditutup bersama isolate-nya, untuk menjaga jumlah file descriptor.

Yang hilang dan penggantinya

- **RLS** tidak ada. Isolasi tenant digantikan file per tenant (lebih kuat). Pengaman per baris di dalam aplikasi tenant digantikan `db.scoped({ user_id })` di SDK, Cedar di guard dan action, serta test otorisasi wajib untuk modul resmi.
- **LISTEN/NOTIFY** tidak dibutuhkan: semua request tenant berada di satu node, pub/sub realtime cukup in-process.
- **Logical replication** untuk realtime data digantikan `update_hook` yang memberi tahu tabel dan rowid yang berubah.

Jalur upgrade ke Postgres

Tenant yang melewati batas tulis atau ukuran dipindahkan ke Postgres. Karena modul hanya menulis dialek SQLite (D18), modul resmi dipelihara dengan test ganda, dan modul pihak ketiga harus menandai diri "Postgres-ready" untuk dipakai di tenant Postgres. Ini pembatasan yang diputuskan sejak awal.

8. Struktur aplikasi dan routing

URL dan perilaku ditentukan oleh lokasi dan nama file. Konfigurasi hanya dipakai sebagai jalan keluar.

```
my-app/
├── renox.toml              config app + daftar modul
├── public/                 static, di-serve Rust di luar kuota tenant
├── pages/                  SSR
│   ├── _layouts/main.html  layout (berawalan _ bukan route)
│   ├── _404.html
│   ├── index.html          /
│   └── products/
│       ├── index.html      /products
│       ├── index.ts
│       ├── [id].html       /products/:id
│       └── [id].ts
├── partials/               komponen template (<x-nama>)
├── functions/              API: functions/api/checkout.ts -> /api/checkout
├── migrations/             SQL milik app
├── policies/               Cedar milik app
└── overrides/<modul>/      menimpa template modul tanpa fork
```

Pola	Contoh	URL
Statis	`pages/about.html`	`/about`
Index folder	`pages/products/index.html`	`/products`
Parameter	`pages/products/[id].html`	`/products/42` → `params.id`
Catch-all	`pages/docs/[...slug].html`	`/docs/a/b` → `params.slug`
API	`functions/api/orders.ts` dengan export `GET` / `POST`	`/api/orders`
WebSocket	`functions/ws/chat/[room].ts` dengan export `ws`	`/ws/chat/:room`

Prioritas kalau bentrok: statis → parameter → catch-all. Bentrokan ambigu (dua modul me-mount path sama) ditolak saat deploy.

Pasangan `.ts` dan `.html` tidak wajib

File	Perilaku
`.html` saja	Template dirender tanpa loader
`.ts` saja	`load()` mengembalikan `Response`
`.ts` + `.html`	`load()` mengembalikan object → data template

```
// pages/products/[id].ts
import type { LoaderContext } from "renox";

export const guard = { action: "product.read" };

export async function load({ params, db }: LoaderContext) {
  const product = await db.queryOne("select * from shop__products where id =
?", [params.id]);
  if (!product) return new Response(null, { status: 404 });
  return { product, inStock: product.stock > 0 };
}

export const actions = {
  async addToCart({ form, db, session }) { /* ... */ },
};
```

Alasan dua file: editor memberi syntax highlight dan type checking TS tanpa plugin, dan transpiler lebih sederhana. Format satu file (`<script server>`) bisa ditambahkan belakangan sebagai gula sintaks.

```
[[routes]]
path = "/p/:id"
page = "products/[id]"     # alias URL lama
```

9. Template engine

Template adalah HTML valid dengan logika sebagai atribut. Saat deploy, sintaks atribut di-transpile menjadi source minijinja, lalu di-compile dan di-cache. Saat request, Rust hanya mengeksekusi template yang sudah di-compile. Sintaks tidak memengaruhi kecepatan; yang menentukan adalah compiler dan marshaling data.

```
<r-layout use="main">
  <ul id="product-list" r-fragment="product_list">
    <li r-for="p in products" class="card" :class="{ 'card--empty': p.stock
== 0 }">
      <a href="/products/{{ p.id }}">{{ p.name }}</a>
      <span r-if="p.stock == 0">Stok habis</span>
      <span r-else>Rp {{ p.price | rupiah }}</span>
      <button r-can="cart.add" :resource="p" hx-
post="?/addToCart">Tambah</button>
    </li>
    <li r-empty="products">Produk tidak ditemukan</li>
  </ul>
</r-layout>
```

Kebutuhan	Sintaks	Hasil transpile
Interpolasi (auto-escape)	`{{ user.name }}`	sama
Filter	`{{ p.price | rupiah }}`	sama
HTML mentah	`r-html="post.body"`	`{{ post.body | safe }}`
Kondisi	`r-if`, `r-else-if`, `r-else`	`{% if %}`
Loop	`r-for="p in items"`	`{% for %}`
List kosong	`r-empty="items"`	`{% else %}` dalam `for`
Atribut dinamis	`:href`, `:class="{ active: x }"`	ekspresi atribut
Fragment	`r-fragment="nama"`	`{% block nama %}`
Komponen	`<x-product-card :product="p">`, `<slot />`	macro, di-inline saat compile
Layout	`<r-layout use="main">`	`{% extends %}`
Slot antar modul	`<r-slot name="admin.sidebar" />`	include dari registry slot
Izin	`r-can="action" :resource="x"`	`{% if can(...) %}` (bagian 12)

Atribut `r-*` dan `:*` dihapus dari output; `hx-*` dan `x-*` dibiarkan. Interpolasi memakai `{{ }}` karena kurung tunggal bentrok dengan `x-data="{ open: false }"` dan `hx-vals='{...}'`, dan tidak diproses di dalam `<script>` dan `<style>`.

Bahasa ekspresi dibatasi

Hanya akses properti, literal, perbandingan, `&&`, `||`, `!`, `??`, dan filter. Tidak ada function call bebas, assignment, atau side effect. Logic yang tidak muat di template dipindahkan ke loader TS. Karena template adalah DSL tanpa eksekusi kode, template tenant aman dievaluasi di thread Rust.

Di mana waktu render habis

Untuk halaman 20–30 KB dengan 100 item (perkiraan, harus diukur):

Tahap	Biaya
Parse dan compile	0 di jalur request (saat deploy)
Eksekusi instruksi	~20–100 µs
Escaping (SIMD) + buffer	~15–40 µs
Marshaling data V8 → Rust (`serde_v8`)	**~50–300 µs**

Biaya terbesar bukan di engine, melainkan di marshaling. Pembanding: React `renderToString` 2–10 ms, Svelte SSR 0,5–2 ms, Hono JSX 0,3–1 ms.

Dua jalur render, dipilih lewat benchmark (D7)

	A. Render di Rust (minijinja)	B. Template di-compile jadi fungsi JS di isolate
Marshaling	Ada	Tidak ada
Beban CPU dan heap tenant	Tidak dihitung ke kuota tenant	Dihitung
Unggul saat	Data kecil–menengah, fragment	Data sangat besar

Untuk hosting multi-tenant, jalur A punya keunggulan non-teknis: render dikerjakan platform, isolate tenant selesai begitu loader mengembalikan data. Jalur B tetap disediakan untuk halaman berdata besar.

Optimasi dan batas

- Potongan statis digabung menjadi satu string; halaman 500 node hanya puluhan instruksi.
- Request fragment hanya merender satu block; loader diberi tahu fragment mana yang diminta.
- Marshaling langsung ke `minijinja::Value` tanpa JSON perantara.
- Streaming `<head>` sebelum loader selesai; cache fragment dengan key hash data + varian izin.
- Template tenant bisa menjadi vektor DoS (loop bersarang 10.000 × 10.000 di thread Rust). Wajib: `fuel` minijinja, batas iterasi, batas ukuran output, batas kedalaman include, timeout render yang dihitung ke kuota tenant.

Opsi kedua: TSX

Loader boleh mengembalikan JSX yang dirender di isolate. Untuk tim yang lebih suka gaya React. Memakai kuota tenant, sehingga bukan default.

10. Partial rendering dan realtime

Untuk update sebagian halaman, server mengirim fragment HTML dan browser hanya menukar bagian yang berubah. Satu template dan satu loader melayani render penuh maupun fragment.

Alur fragment

1. Request pertama ke `/products` dirender penuh.
2. htmx mengirim `GET /products?q=sepatu` dengan `HX-Request: true` dan `HX-Target: product-list`.
3. Rust menjalankan loader yang sama (diberi tahu fragment yang diminta), lalu merender hanya block `product_list` lewat render per block minijinja.
4. htmx men-swap fragment dengan ekstensi `morph` (idiomorph), dibungkus View Transitions.

Action yang meng-update beberapa bagian sekaligus (tombol dan badge keranjang) mendapat fragment tambahan dengan `hx-swap-oob="true"` otomatis.

Kenapa smooth: morphing hanya mengubah node yang berbeda, sehingga focus, scroll, media, dan animasi CSS tidak ter-reset; View Transitions menganimasikan perubahan state antar fragment.

Kebutuhan	Solusi	Dikerjakan oleh
Halaman pertama	SSR penuh	Rust + loader
Filter, paging, form	Fragment + morph	Rust render, htmx swap
Dropdown, modal, tab	Alpine.js / vanilla	Browser, tanpa server
Widget sangat interaktif	Islands (Preact/Svelte), di-bundle saat deploy	Browser
Notifikasi, feed	SSE	Rust
Chat, kolaborasi	WebSocket	Rust

Realtime: SSE dan WebSocket

Koneksi dipegang Rust di cell tenant, bukan oleh isolate. Isolate hanya dipanggil saat connect, pesan masuk, dan disconnect, sehingga ribuan koneksi idle tidak memakai heap tenant. Biaya per koneksi beberapa KB di tokio.

```
// functions/ws/chat/[room].ts  ->  wss://host/ws/chat/:room
export const ws = {
  async open({ conn, params, auth, realtime }) {
    await auth.requireUser(conn.req);
    realtime.subscribe(conn, `room:${params.room}`);
  },
  async message({ conn, data, params, realtime, render }) {
    const html = await render("partials/chat-message.html", { msg: data,
user: conn.user });
    realtime.publish(`room:${params.room}`, html);
  },
};
```

```
<div hx-ext="sse" sse-connect="/sse/orders" sse-swap="order-updated">…</div>
```

- `realtime.publish(topic, payload)` bisa dipanggil dari handler WebSocket, action, service, atau event handler durable. Karena semua request tenant berada di satu cell, pub/sub cukup in-process (`tokio::sync::broadcast`). Lintas node hanya untuk tenant tier replikasi, lewat transport yang pluggable (NATS/Redis).
- Realtime dari perubahan data memakai `update_hook` SQLite: tabel dan rowid yang berubah dipetakan ke topic, difilter oleh policy.
- Keamanan: autentikasi saat connect dengan cookie session, otorisasi per topic lewat hook `realtime.authorize`, heartbeat, batas koneksi per user dan ukuran pesan, backpressure (client lambat diputus), SSE mendukung `Last-Event-ID`.
- Koneksi realtime dimeter per tenant (jumlah koneksi dan pesan) untuk billing.

11. Sistem modul dan komunikasi antar modul

Modul adalah mini app dengan struktur folder yang sama seperti app, diproses oleh loader yang sama. Modul resmi (auth, 2fa, billing, admin, storage) adalah produk utama Renox: alasan tenant memilih platform ini.

```
modules/billing/
├── module.toml     manifest
├── migrations/     SQL (dialek SQLite), tabel billing__*
├── pages/  functions/  partials/  public/
├── policies/       Cedar default
├── services.ts     kontrak publik untuk modul lain
└── hooks.ts        event handler, hook, middleware
```

```
[module]
name = "billing"
version = "1.2.0"

[dependencies]
auth = "^1.0"

[config]
provider = { type = "enum", values = ["stripe", "midtrans", "xendit"] }

[permissions]
network = ["api.midtrans.com"]
tables = "billing__*"

[exports]
views = ["billing__public_user_plans_v1"]

[[slots]]
name = "admin.sidebar"
template = "partials/admin-menu.html"
requires = "invoice.read"
```

```
# renox.toml (app tenant)
[modules]
auth    = { version = "^1.0" }
billing = { version = "^1.2", mount = "/billing", config = { provider =
"midtrans" } }
reports = { registry = "acme/reports", version = "^0.4", isolation =
"isolate" }
```

Saat deploy: resolve versi dan tulis `renox.lock`, bangun graph dependency (siklus ditolak), jalankan migration urut topologis, gabungkan route dengan prefix `mount` (konflik ditolak), daftarkan services, event, hook, dan slot ke Module Broker.

Mode isolasi modul (D12)

Mode	Cara kerja	Biaya panggilan	Untuk
`shared` (default)	Semua modul di-bundle ke satu context di isolate tenant	Panggilan fungsi JS biasa, tanpa serialisasi	Modul milik tenant dan modul resmi
`context`	Context terpisah dalam isolate tenant	Mikrodetik + serialisasi	Modul yang butuh global terpisah
`isolate`	Isolate sendiri dengan heap limit, di proses sandbox tenant	Lebih mahal, lintas isolate	Modul marketplace

Apa pun modenya, izin tabel ditegakkan SQLite authorizer dan izin network oleh op `fetch`, bukan oleh konvensi JS.

Lima mekanisme komunikasi

Mekanisme	Sifat	Butuh dependency	Dipakai saat
Service call	Sinkron	Ya	Butuh jawaban sekarang
Event	Asinkron, fire-and-forget	Tidak	Mengumumkan sesuatu sudah terjadi
Hook	Sinkron, berurutan, bisa menghentikan alur	Tidak	Ikut menentukan proses modul lain
Public view	Read-only SQL, diberi versi	Ya	Join lintas modul tanpa N+1
Slot	Penyisipan template	Tidak	UI di halaman modul lain

```
// modules/billing/services.ts
export async function getActivePlan(ctx, userId: string): Promise<Plan |
null> { /* ... */ }

// pemanggil
import { billing } from "renox:modules";
const plan = await billing.getActivePlan(user.id);
```

Broker memastikan dependency dideklarasikan, meneruskan `ctx` milik modul tujuan (db dengan authorizer modul itu, config, user, transaksi, request_id, deadline), dan meng-generate `renox-modules.d.ts` untuk type checking.

```
// modules/2fa/hooks.ts
export const hooks = {
  "auth.after_login": { priority: 10, async handle(ctx, { user }) {
    if (await isEnabled(ctx, user.id)) return { halt: true, response:
ctx.redirect("/2fa/verify") };
  }},
};
export const on = {
  "billing.invoice_paid": async (ctx, e) => ctx.mail.send(e.userId, "invoice-
paid", e),
};
```

Event durable (D14)

	In-process	Durable (default)
Eksekusi	Langsung, di request yang sama	Setelah commit, oleh worker cell
Handler gagal	Request gagal	Retry exponential backoff, lalu dead-letter
Server restart	Event hilang	Tetap terkirim
Jaminan	Tepat sekali dalam request	At-least-once; handler idempotent dengan `e.id`

Baris event ditulis ke `renox__outbox` di transaksi yang sama dengan data. Worker dibangunkan oleh `update_hook`, bukan polling. Transaksi lintas modul memakai satu koneksi penulis tenant: kalau `inventory.reserve` gagal, order dan invoice ikut di-rollback.

Override dan migration

- Template modul bisa ditimpa di `overrides/<modul>/...`; logic tidak. Hash file asli disimpan dan diff ditampilkan saat upgrade.
- Migration forward-only dengan expand/contract (D17). Public view diberi versi (`_v1`), versi lama dipertahankan minimal satu rilis mayor.

Aturan

1. Service call dan public view wajib dependency eksplisit; siklus ditolak saat deploy.
2. Modul hanya menulis ke tabel prefix-nya sendiri; data modul lain lewat services, events, atau view.
3. Nama event, signature service, hook, slot, dan view adalah public API modul, di-versioning dengan semver. `renox check` mendeteksi kontrak yang hilang.

12. Otorisasi: RBAC, ABAC, dan relasi

Ada dua jenis permission yang berbeda. Permission modul (capability: modul ini boleh akses apa) ditentukan `module.toml` dan ditegakkan sandbox, authorizer, dan op Rust. Permission user (otorisasi: user ini boleh melakukan apa) ditentukan policy dan role milik aplikasi tenant, dan dievaluasi oleh satu policy engine di Rust untuk semua modul. Bagian ini membahas yang kedua.

Model	Dipakai untuk	Contoh
RBAC	Mayoritas izin, dikelola lewat UI admin	Role `finance` boleh membaca dan me-refund invoice
ABAC	Aturan berbasis atribut user, resource, konteks	Refund di atas Rp 10 juta hanya `finance_manager`; aksi sensitif wajib 2FA
ReBAC	Kepemilikan dan keanggotaan	Pemilik dokumen, anggota organisasi, admin tim

Cedar (D15)

Policy ditulis dengan Cedar dan dievaluasi `cedar-policy` di balik trait `PolicyEngine`. Native Rust, policy di-load saat runtime, RBAC/ABAC/relasi dalam satu bahasa, `forbid` menang atas `permit`, dan policy divalidasi terhadap schema saat deploy.

```
# modules/billing/module.toml
[authz]
actions = ["invoice.read", "invoice.create", "invoice.refund"]
sensitive = ["invoice.refund"]
[authz.entities]
Invoice = { attrs = ["amount", "org", "status"] }     # hanya field ini yang
dikirim ke engine
[authz.default_roles]
finance = ["invoice.read", "invoice.refund"]
```

```
permit (principal in Role::"finance",
        action in [Action::"invoice.read", Action::"invoice.refund"],
        resource is Billing::Invoice)
when { resource.org == principal.org };

forbid (principal, action == Action::"invoice.refund", resource)
when { resource.amount > 10000000 } unless { principal in
Role::"finance_manager" };

forbid (principal, action in Action::"sensitive", resource)
unless { context.mfa_verified };
```

Assignment role disimpan di `renox__role_assignments (user_id, role, scope_type, scope_id)` di file tenant. Template selalu bertanya tentang action, bukan role; pemetaan role ke action hanya ada di policy.

Titik penegakan

Lapisan	Cara	Peran
Route	`export const guard = { action: "invoice.read" }`	Menolak sebelum handler; berlaku juga untuk request fragment
Aksi	`await ctx.authz.require("invoice.refund", invoiceDariDB)`	Pemeriksaan ABAC dengan resource dari database, bukan dari client
Data	`db.scoped({ user_id })` + deklarasi kolom pemilik di manifest	Pengganti RLS untuk filter kepemilikan
UI	`r-can="invoice.refund" :resource="inv"`	Hanya menyembunyikan; bukan pengaman
Realtime	Hook `realtime.authorize` per topic	Mencegah subscribe ke topic orang lain

Di template

- `r-can` tanpa resource: lookup O(1) di set action yang dihitung sekali per request. Tidak menyentuh Cedar saat render.
- `r-can` dengan resource: evaluasi Cedar per item (mikrodetik), di-cache per (action, resource.id) dalam request, dihitung sebagai fuel render. Subtree yang ditolak tidak dirender.

- Alternatif untuk halaman besar: `authz.canBatch("invoice.refund", invoices)` di loader, lalu `r-if="inv.canRefund"`.
- Atribut dan action yang tidak dideklarasikan menjadi compile error saat deploy. Resource tanpa field wajib menghasilkan `deny` dan peringatan di dev.
- Cache fragment menyertakan hash set action user; fragment dengan `r-can` berbasis resource tidak di-cache lintas user.
- Slot modul lain dievaluasi dengan action milik modul pemilik slot (`requires` di manifest).

Konteks lintas modul dan audit

- Setiap modul menegakkan policy untuk resource miliknya sendiri; identitas user ikut di setiap service call.
- Event handler, cron, dan job berjalan sebagai `System::"<modul>"` dengan izin eksplisit. Impersonasi admin dicatat.
- Keputusan untuk action `sensitive` dicatat append-only: siapa, aksi, resource, hasil, policy penentu.
- `renox authz explain --user budi --action invoice.refund --resource inv_123` dan kasus uji policy di `renox test`.

Lapisan keamanan terkait

Area	Desain
Autentikasi	Modul `auth`: argon2id, OAuth/OIDC, magic link; session cookie `HttpOnly` `Secure` `SameSite=Lax`; API key/JWT untuk mesin
2FA	Modul `2fa`: TOTP dan WebAuthn, lewat hook `auth.after_login` dan `context.mfa_verified`
Rate limiting	Per IP, user, route di gateway; ketat untuk login dan reset password
Validasi input	Schema validation di action dan function
CSRF, header	Token untuk semua action non-GET termasuk htmx; CSP, `X-Content-Type-Options`, `Referrer-Policy`
Audit	Append-only untuk login, perubahan role, aksi sensitif, deploy, perubahan secret

13. Developer experience

Tenant hanya butuh satu CLI dan editor. Mode dev menjalankan data plane yang sama dengan produksi secara lokal, dengan SQLite, sehingga tidak ada perbedaan perilaku antara laptop dan server.

Perintah	Fungsi
`renox new <nama>`	Project dari template
`renox dev`	Server lokal: hot reload, error overlay, generate type
`renox add <modul>`	Menambah modul, memperbarui `renox.lock`, menampilkan permission yang diminta
`renox migrate`	Migration semua modul urut dependency
`renox check`	Type check TS, validasi template dan policy, konflik route, kontrak modul
`renox test`	Test TS di runtime produksi, termasuk kasus uji policy dan snapshot template per role
`renox deploy`	Bundle, upload, validasi, aktivasi; `--rollback <hash>`
`renox logs --follow`	Log tenant
`renox authz explain`	Menjelaskan keputusan policy

Dev loop

1. File watcher mendeteksi perubahan.
2. Hanya file yang berubah yang diproses: TS di-transpile (swc), template di-transpile ke minijinja dan di-compile ulang.
3. Isolate dibuang dan dibuat baru dari snapshot (D19). Koneksi SQLite tetap hidup.
4. Browser di-refresh lewat SSE; kalau hanya template yang berubah, cukup fragment yang di-swap.

Target dari save sampai browser ter-update: di bawah 300 ms untuk satu file. Membuang isolate memakan puluhan milidetik dan menghilangkan risiko memory leak dari hot-swap context.

Type safety

`renox dev` meng-generate `.renox/types/`: `LoaderContext`, `params` per route, services setiap modul, payload event, dan nama action policy. Type checking di editor lewat `tsc`/tsgo; runtime tidak melakukan type check supaya reload tetap cepat.

Debugging

Source map TS → JS disertakan di bundle dan dipetakan oleh runtime Deno, sehingga stack trace menunjuk ke baris TS asli. Error dari op Rust (query, fetch, render) dibungkus menjadi error JS dengan konteks (nama tabel, host, nama template dan baris). Di dev, elemen yang disembunyikan `r-can` ditampilkan sebagai placeholder dengan nama policy penyebabnya.

Observability

Tracing OpenTelemetry per request: span untuk loader, setiap service call, query, render, dan fetch keluar. Metric per tenant dan per route: latency, error rate, CPU ms, heap, egress. Log terstruktur selalu menyertakan `tenant_id`, `request_id`, dan nama modul.

14. Tinjauan kritis dan jawabannya

Versi 1 dokumen ini diuji dengan tinjauan kritis. Sebagian poinnya tepat dan mengubah desain; sebagian keliru secara teknis. Keduanya dicatat supaya keputusan di bagian 3 bisa dilacak asalnya.

#	Kritik	Penilaian	Jawaban di versi 2
1	Scope masif: membangun runtime, parser, bundler, event bus, authz sendiri	Valid, paling penting	Runtime dari Supabase Edge Runtime (D3), template ke minijinja (D8), parser `html5ever`, policy Cedar; usaha dipusatkan ke control plane dan modul (D20)

#	Kritik	Penilaian	Jawaban di versi 2
2	Overhead serialisasi JS ↔ Rust menghapus keuntungan Rust	Sebagian valid	Untuk query, biaya `serde_v8` setara parsing driver `pg` di Node. Untuk render, marshaling nyata; D7 diputuskan lewat benchmark dengan jalur B sebagai alternatif. Modul sendiri memakai mode `shared` tanpa serialisasi (D12)
3	npm rusak tanpa Node	Valid, akar masalahnya `deno_core` mentah	Deno runtime memberi `npm:` dan `node:*`; dependency dibekukan saat deploy; pekerjaan berat (gambar, PDF) disediakan sebagai API platform
4	Loop berat memblokir modul lain di satu isolate	Valid, perbandingannya keliru	Node juga single-thread per proses. Di Renox: isolate per tenant, watchdog CPU, isolate di-recycle setelah terminate, scheduler tidak mengirim request ke worker sibuk, `ctx.offload()` untuk pekerjaan berat
5a	Migration dan rollback lintas schema rapuh	Valid	Forward-only, expand/contract, view berversi (D17)
5b	`set_config` + PgBouncer merusak performa	Sebagian besar keliru	`set_config(..., true)` lokal per transaksi dan bisa di-pipeline. Tidak relevan lagi: data tenant pindah ke SQLite (D5)
6a	Stack trace membingungkan	Valid sebagai pekerjaan, bukan blocker	Source map ditangani runtime Deno; error op Rust dibungkus dengan konteks

#	Kritik	Penilaian	Jawaban di versi 2
6b	V8 tidak suka context sering dibuat ulang	Dilebih-lebihkan	Dev mode membuang isolate, bukan context (D19)
7	Saran: sidecar Bun/Node via IPC	Ditolak	Bertentangan dengan tujuan hosting: tanpa isolate, 5–10x kurang padat, dan overhead IPC lebih besar dari FFI
8	Saran: SQLite	Diterima, dengan alasan berbeda	Bukan karena "lebih sederhana", tapi karena isolasi per file dan biaya idle nol untuk multi-tenant (D5)

Pertanyaan yang paling menentukan: framework atau platform?

Sekitar 80% nilai desain ini (modul, konvensi, template, batteries) bisa dibangun sebagai framework TypeScript di atas Bun tanpa Rust, dengan usaha 3–5x lebih kecil. Rust hanya dibenarkan oleh satu hal: menjalankan kode orang lain dengan sandbox yang padat. Karena tujuan bisnisnya adalah hosting berbayar, Rust di data plane dipertahankan (D1, D2), dan Bun dipakai di control plane (D4).

Bun vs Rust + V8 untuk hosting

Model	Memory per tenant idle	Cold start	Isolasi	npm	Dipakai oleh
Bun, proses per tenant + cgroups	~30–50 MB	~10–30 ms	Kuat (OS)	Terbaik	Pola Fly.io
Bun di microVM	~100 MB+	~100 ms+	Sangat kuat	Terbaik	AWS Lambda
Rust + V8 isolate per tenant	~2–10 MB	~1–10 ms	Cukup, dilapisi (D6)	Baik via Deno	Cloudflare, Deno Deploy, Supabase
Wasm per tenant	< 1 MB	< 1 ms	Sangat kuat	Buruk untuk JS	Fermyon, Fastly

Performa per request V8 dan JavaScriptCore kurang lebih setara. Yang membedakan adalah kepadatan (5–10x) dan kontrol batas per tenant, dan itulah yang menentukan margin hosting.

15. Risiko dan mitigasi

Risiko terbesar bukan di Rust, melainkan di keamanan sandbox (karena itu produknya), di kompatibilitas npm, dan di godaan membangun platform sebelum ada produk.

Risiko	Dampak	Mitigasi
Escape dari isolate (bug V8, side-channel)	Kebocoran data antar tenant; kematian bisnis	Isolasi berlapis (D6), update V8 cepat lewat rilis Deno, tier proses, bug bounty setelah ada pelanggan
SSRF ke infrastruktur internal	Akses ke control plane dan metadata cloud	Blok IP privat dan metadata endpoint di op `fetch`, allowlist per tenant
Template tenant sebagai DoS di thread Rust	Satu tenant memakan CPU cell	Fuel, batas iterasi dan output, timeout render yang dihitung ke kuota
Library npm tidak jalan di Deno	Tenant pergi	Daftar kompatibilitas publik, uji paket populer (Stripe, OAuth, email) sejak F1, API platform untuk gambar/PDF
Marshaling V8→Rust menghapus keuntungan render	Klaim performa tidak terbukti	Benchmark di F2 dengan jalur B sebagai alternatif (D7)
Node cell mati	Tenant di node itu down	Litestream ke S3 untuk tier dasar (RPO detik), replika sinkron untuk tier tinggi, artifact di object storage
Satu penulis per file SQLite	Tenant sibuk mentok	Batas tier, metrik lock wait, jalur upgrade ke Postgres
Global state bocor antar request di context `shared`	Data user tercampur dalam satu tenant	Data per request lewat `ctx`, peringatan di dev, dokumentasi modul
Kontrak modul berubah tanpa pemberitahuan	App tenant rusak setelah upgrade	Semver, type yang di-generate, `renox check`, view berversi
Override template tertinggal versi	Tampilan rusak	Hash file asli dan diff saat upgrade

Risiko	Dampak	Mitigasi
Ukuran binary dan build V8	CI lambat, binary 50–100 MB	Prebuilt `rusty_v8`; build hanya saat platform berubah
Outbox dan log membengkak	Disk cell penuh	Pembersihan berkala, dead-letter, batas volume log per tenant
Scope merayap kembali	Platform tidak pernah dipakai	Roadmap bertahap, kriteria berhenti (bagian 16), dogfooding
Bus factor satu orang	Proyek mati saat pembuatnya sibuk	Dokumen ini, komponen dari ekosistem yang hidup, modul resmi yang kecil

16. Roadmap, kriteria berhenti, dan pertanyaan terbuka

Setiap fase menghasilkan sesuatu yang bisa dijalankan dan mempunyai kriteria selesai. Dua produk nyata dibangun berbarengan dengan platform sejak F1, dan menjadi pelanggan pertama.

1. **F0: Spike (2–3 minggu).** Jalankan Supabase Edge Runtime secara lokal, deploy satu function TS, ukur memory per isolate dan cold start; uji paket npm yang dibutuhkan produk pertama (Stripe/Midtrans, OAuth, email); prototipe transpiler atribut → minijinja; benchmark marshaling vs render.
   - Selesai kalau: angka memory, cold start, dan kompatibilitas memenuhi target. Kalau tidak, arah dievaluasi ulang sebelum menulis kode platform.
2. **F1: Data plane minimum.** Gateway, routing per tenant, isolate per deployment, batas CPU/memory/wall-clock, `fetch` dengan allowlist, SQLite per tenant dengan authorizer, static serving.
   - Selesai kalau: dua tenant berjalan di satu node, satu tenant yang infinite loop tidak mengganggu yang lain, dan metering tercatat.
3. **F2: SSR dan template.** Transpiler atribut, loader, actions, layout, komponen, fragment htmx, fuel dan batas render.
   - Selesai kalau: contoh toko (list, detail, form, search live) berjalan, dan benchmark D7 diputuskan.
4. **F3: Control plane minimum (Bun).** API deploy, artifact store, route table, secret, dashboard sederhana, log, billing dari meter.
   - Selesai kalau: `renox deploy` dari laptop ke server hidup dalam < 30 detik, rollback bekerja.

5. **F4: Modul.** Manifest, resolver, migration forward-only, services, events (outbox + `update_hook`), hook, slot, override; modul `auth` sebagai modul pertama.
   - Selesai kalau: produk pertama memakai modul `auth` tanpa kode auth sendiri.
6. **F5: Otorisasi dan realtime.** Cedar, `guard`, `authz.require`, `r-can`, `db.scoped`; SSE/WebSocket dengan pub/sub in-process; modul `2fa` lewat hook.
   - Selesai kalau: refund ditolak untuk user tanpa role atau tanpa 2FA, dan chat room berjalan.
7. **F6: Keandalan dan keamanan produksi.** Proses sandbox (seccomp, cgroup), tier proses, Litestream, replika untuk tier tinggi, OpenTelemetry, audit log, blok SSRF, pengujian penetrasi internal.
   - Selesai kalau: node dimatikan paksa dan tenant pulih di node lain tanpa kehilangan data lebih dari RPO tier-nya.
8. **F7: Pelanggan luar.** Modul `billing` dan `admin`, registry modul dengan tanda tangan, mode `isolate` untuk modul marketplace, dokumentasi publik, onboarding.

Kriteria berhenti

- Setelah F0: memory per isolate > 20 MB, cold start > 100 ms, atau paket npm inti tidak jalan → pertimbangkan Bun proses per tenant untuk tier berbayar saja, atau hentikan.
- Setelah F4 (target 4–5 bulan dari mulai): belum ada dua produk nyata yang berjalan di Renox → hentikan platform, simpan produk.
- Setelah F7 (target 9–12 bulan): belum ada tenant berbayar dari luar → Renox menjadi alat internal saja, tanpa investasi marketplace.

Pertanyaan terbuka

- Target perangkat per node: 8 GB atau 16 GB? Menentukan jumlah isolate warm dan tier.
- Harga: per request, per CPU-ms, atau flat per tenant dengan kuota? Model Cloudflare (request + CPU-ms) paling selaras dengan meter yang ada.
- Apakah tenant boleh membawa Postgres sendiri sejak awal, atau baru di tier atas?
- Region kedua: kapan, dan apakah dengan replikasi SQLite atau tenant terikat ke satu region?
- Auth: dibangun sendiri sebagai modul, atau kompatibel dengan standar yang ada (OIDC provider) supaya tenant bisa memakai identitas eksternal?
- Lisensi data plane: open source (menarik kontributor dan kepercayaan) atau tertutup (melindungi bisnis hosting)?